



开源语音智能体全解

Hugging Face speech-to-speech: 让机器实时地「听·想·说」

VAD → STT → LLM → TTS 级联管线 · 全模块可换 · 兼容 OpenAI Realtime 协议 · Apache-2.0
源码版本 v1.0.0 (commit 81b688b) · 已投产为数千台 Reachy Mini 机器人的对话后端

⚡ 低延迟

✖ 768 种组件组合

🏠 可全本地部署

🤖 机器人同款

一·十二问图解

Q1 这是什么？"实时听、想、说"到底解决什么问题？

一句话：**它是一套把「听见你 → 想明白 → 说给你」串成实时流水线的开源框架**，Hugging Face 官方出品。你说一句话，它像真人一样接住并开口回答——中间的语音识别、大模型、语音合成全部用开源模型，可以整套跑在你自己电脑上，**一个字都不上云**。

没有它之前：接 OpenAI 的 Realtime API——效果爽，但①按量付费②对话音频必须传到人家服务器③模型黑盒不可换。这个项目给出**开源平替**：同样的接法（下文 Q8），后端换成自己的。



Q2 和文本聊天机器人比，实时语音难在哪三件事？

文本聊天是"发一条、回一条"，节奏由人控制。语音是**连续流**，机器必须自己回答三个文本时代不存在的问题：

难题	本质	本项目对策
① 你在说话吗？	麦克风里全是声音（包括噪音），先分清"话"和"不是话"	VAD 语音活动检测 (Q4)
② 你说完了吗？	"呃.....让我想想"中间的停顿不是说完	Smart Turn 轮末判断 (Q5)
③ 等得起吗？	对话停顿超过 ~1.5 秒人就尴尬，响应必须 <1 秒级	流式接力管线 (Q7) + 打断 (Q6)

这三件事全发生在"话还没变成文字"的音频世界里——这就是语音智能体和聊天机器人 Coding 上的分水岭。

Q3 核心架构：为什么是 8 条队列 + 4 段线程？

打开源码 `s2s_pipeline.py`，你会看到 8 条 `queue.Queue` 把 4 类 handler 串成流水线，每段一个线程、各跑各的：

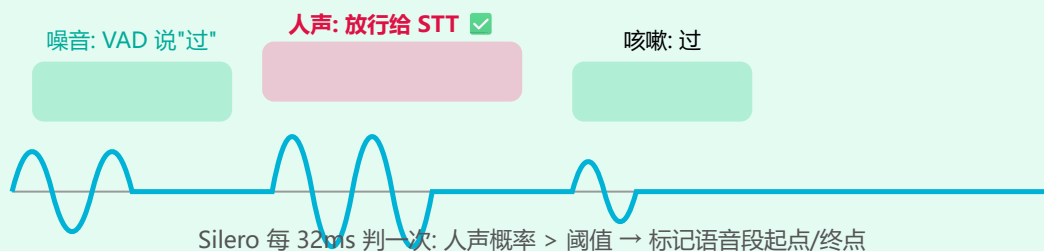
```
麦克风音频流 → [VAD线程] → spoken_prompt → [STT线程] → text_prompt
→ [LLM线程] → lm_response → [TTS线程] → 扬声器音频流
    ↘ text_output (字幕/日志)
```

队列解耦 = 各段用自己的节奏干活：VAD 按 32ms 小块处理，LLM 慢悠悠吐 token，TTS 一小块一小块合成——谁也不等谁，全靠队列缓冲衔接。每段 handler 都实现同一个 `BaseHandler` 接口（输入队列 → `process()` 生成器 → 输出队列），所以**换掉任何一段，其余纹丝不动**。这和知识库里的 `[[Agent循环]]` 的“感知 → 决策 → 行动”是同一思想的工程化：只不过每个箭头都换成了线程安全的队列。



Q4 VAD：机器怎么知道你在说话？

VAD (Voice Activity Detection, 语音活动检测) 一句话解释：给每一小段音频打个“这是不是人声”的分数。本项目默认用 **Silero VAD**——一个不到 2MB 的小神经网络，每 32ms 出一次结论，快得几乎零负担。为什么必须有它？因为 STT 模型不便宜：如果麦克风里 24 小时的噪音全喂给 Whisper 级别的模型，电费和延迟都爆炸。**VAD 是守门员：只把“有人在说话”的片段放行给后面的重型模型**。可选后端两个：Silero（默认，快）和 FireRed（新式流式 VAD）。



Q5 说完没完？Smart Turn 为什么必须存在

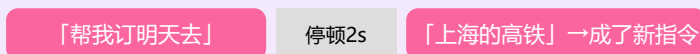
这是全项目**最精妙的设计**。VAD 只能回答"声音停了"，回答不了"话说完了没"。你想想这个场景：

🗣️「帮我订明天去（**想了两秒**）上海的高铁」

纯 VAD 在那两秒停顿就把前半句提交给 LLM 了——机器人开始回复"订去哪？"，**你被自己的机器人打断了**。所以项目在 VAD 后面挂了第二级判断：**Smart Turn v3.2**（来自 pipecat-ai 的开源 ONNX 模型），听完整句的声学

与语义特征再判 complete / incomplete。
分层策略：**Silero 管"快"（32ms 级发现停顿），Smart Turn 管"准"（发现停顿后再花零点几秒裁决）**。没说完就 hold 住继续听（最长等 2 秒强制提交）。我的实验 exp2 复刻了这套状态机：犹豫句纯 VAD 提交 2 次且拦腰截断，加 Smart 判断后恰好 1 次完整提交（见实验部分）。

❌ 纯静音计时：



↑ 这里就提交了！

✅ +Smart Turn:



Q6 打断 (barge-in)：机器人正说着，你怎么插话？

真人对话的核心特征：**对方没说完你也可以开口**。机器做到这点叫 barge-in。实现机关：麦克风和扬声器**同时在跑**（全双工），VAD 持续监听——一旦在你说话途中检测到新的人声，管线立刻：

1. 停止当前 TTS 播放（丢弃还没播的音频块）
2. 取消正在生成的 LLM 响应（`response.cancel`）
3. 把你的新话当作新一轮输入

源码里 LLM handler 会收到"有新语音进来"的通知（`audio_input_notifier`），立即停止生成——**省掉的每一毫秒算力都是真金白银**，尤其 LLM 在自己机器上跑的时候。担心扬声器声音被麦克风误判成"人声"？加 `--local_audio_block_mic_during_playback`：播放时暂停收音，牺牲打断换来不怕回声。

Q7 流式接力：为什么你能提前 700ms 听到回答？

如果等 LLM 把整句写完再交给 TTS，响应就慢一截。本项目的做法：**LLM 一边吐 token，TTS 一边把凑够一小撮的 token 合成音频，扬声器立刻开播**——三级流水线重叠运行。衡量它的核心指标叫 **ttfa (time to first audio, 首音频延迟)**。

我实验 exp1 实测（文本级模拟）：ttfa 0.892s vs 整句完成 1.614s——**用户提前 722ms 听到声音**。上游文档的真实测量：stt=0.18s / llm=1.24s / tts_ttfa=0.12s / e2e=2.01s，同样是首包远早于全量。



Q8 OpenAI Realtime 协议：为什么要"假装"成 OpenAI？

这是本项目最聪明的一步棋。OpenAI 的 Realtime API 定义了一套 WebSocket/WebRTC 事件协议（我从源码里数出 31 种标准事件名）：`input_audio_buffer.speech_started`、`response.output_audio.delta`、`response.done`.....全世界的客户端 SDK、网页组件、机器人框架都认这套协议。

本项目把整条开源管线包装成一个"讲 OpenAI 方言"的服务端：客户端以为自己连的是 OpenAI，身后其实是 Silero + Parakeet + 你自己的 LLM + Qwen3-TTS。于是官方 demo 里那个著名的演示成立——网页客户端把端点地址从 改成本机地址，**一行业务代码不改**，就从托管服务切到全本地栈。协议事件长这样（exp3 重放并逐个校验过）：

```
C→S input_audio_buffer.append      (你说的话, 音频块流入)
S→C input_audio_buffer.speech_started (VAD: 你开口了)
S→C input_audio_buffer.speech_stopped (VAD+SmartTurn: 说完了)
S→C conversation.item.input_audio_transcription.completed (STT转写)
C→S response.create → S→C response.created
S→C response.output_audio_transcript.delta (文字流)
S→C response.output_audio.delta        (音频块流)
S→C response.done                      (附延迟账本 metadata)
```

类比知识库：`[[LLM工具调用]]` 的统一协议让模型可换，这里同理——**协议面标准化 = 生态位免费继承**。

Q9 768 种组合：全模块化是怎么做到的？

我写脚本直接数了源码（exp4）：VAD 2 种 × STT 12 种 × LLM 4 类 × TTS 8 种 = **768 种管线配置**。全部通过命令行参数切换，例如：

```
speech-to-speech local --stt parakeet-tdt \
  --llm_backend chat_completions \
  --model_name Qwen/Qwen3-32B \      # LLM 指向任意 OpenAI 兼容端点
  --tts qwen3
```

槽位	候选（默认加粗）
VAD	silero · firered
STT	parakeet-tdt · whisper · faster-whisper · paraformer(中文友好) · sensevoice · qwen3-asr · nemo · mlx 系列...
LLM	任意 OpenAI 兼容 API: chat-completions / responses-api / transformers 本地 / mlx-lm → vLLM、llama.cpp、GLM 都能接
TTS	qwen3 · kokoro · chatTTS · facebook-mms(多语) · supertonic · pocket · omnivoice · openai

机关就在 Q3 说的 `BaseHandler` 统一接口 + 注册表（`backend_registry.py`）：新后端只要实现同样的输入/输出类型就能注册进来，**加一个 TTS 不用动其余 767 种组合里的任何一行**。

Q10 延迟账本：每一毫秒花在哪？

服务端每轮对话结束会写一条“延迟账单”（`docs/response-latency.md`），字段精确到阶段：

字段	含义	上游实测样例
<code>vad_decision</code>	VAD+Smart-Turn 判断“说完了”耗时	0.36s（含 hold 0.24s）
<code>stt</code>	最终转写耗时	0.18s
<code>llm</code>	语言模型生成	1.24s（通常大头）
<code>tts_ttfa</code>	首块音频合成延迟	0.12s
<code>e2e</code>	说完→首个音频的总延迟	2.01s

这份账单甚至塞进 `response.done` 事件的 metadata 回传给客户端（≤512 字符的紧凑 JSON，exp4 已验证合规）。**优化的第一步永远是知道钱花在哪**：看到 llm 占 60% 就去换小模型/量化（知识库 `[[量化与 GGUF]]`），看到 vad_decision 大就调 hold 参数。

Q11 级联 vs 端到端：为什么不用 GPT-4o voice 那种一坨模型？

业界另一条路线是"原生语音模型"：音频进、音频出，一个模型全包（如 GPT-4o 语音版、Moshi）。它保真度高（能听出语气、笑声），但：**黑盒、不可换、必须上云、贵**。本项目选级联（cascade）路线，本质是用"模块化"换"端到端智能"，换来的是：

	级联(本项目)	端到端
可换件	✔ 768 种组合，中文 TTS 想换就换	✗ 整坨锁定
隐私	✔ 可全本地，音频不出门	✗ 音频上云
大脑自由	✔ LLM 槽随便插（接入自家知识库/工具调用）	⚠ 受平台限制
副信息	✔ 中间产物全是文字：转写、字幕、日志白送	✗ 只有声音
语气理解	⚠ 转写丢韵律（"嗯....."的犹豫靠 Smart Turn 补）	✔ 原生保留

选型口诀：要"听懂语气"选端到端，要"可控可换可私有"选级联。值得注意的是项目也留了口子——Gemma 4 音频输入示例支持直接听音频的模型，说明两条路线正在合流。

Q12 它和 Reachy Mini 什么关系？无声卡电脑能玩什么？

README 第一页就写明：**这套管线已投产为数千台 Reachy Mini 机器人的对话后端**——正是知识库 **[[WebRTC机器人应用]]** 里那台桌面机器人的"嘴和耳朵"。机器人本体管表情动作（[[Stewart并联平台]]、[[情感运动序列]]），这个项目管对话，通过 WebRTC 把声音送进送出。这也是它把"低延迟"磨到极致的原因：机器人回话慢半拍，就会显得很蠢。

无声卡/无麦克风电脑（比如本机）能玩什么？——语音端到端跑不了，但①整套架构可文本级复刻（本指南 4 个实验全是）；②服务端可以无音频启动，用 WebSocket 客户端喂文字/音频文件；③仓库 60+ 个测试文件大多不依赖声卡。诚实声明：本指南所有延迟数字中，管道级数据来自我的模拟实验，秒级实测数据引自上游文档。

二·动手实验（文本管道级，已全部真跑通）

⚠ **环境限制声明：**本机 Windows 无声卡无麦克风，无法做真实语音端到端。以下实验用文本块/合成信号模拟音频，验证的是架构与算法逻辑，不是声音本身——这正是"没有硬件也能学到骨头"的路径。

实验 1·复刻级联管线：亲眼看流式接力抢跑（主实验）

操作：`cd F:\reminder\s2s-voice-agent\exercise && python mini_s2s.py`

目的：用 4 个线程 + 5 条队列复刻上游 8 队列架构，验证 $t_{tfa} \ll e_{2e}$ 。

实现：FakeVAD→FakeSTT(0.18s)→FakeLLM(逐词 0.12s)→FakeTTS(每 3 词一块, 0.05s)。

✔ 实测输出：`tts_ttfa=0.892s, e2e=1.614s, ttfa 占 55%, 用户提前 722ms 听到声音`——LLM 还在说"南锣鼓巷"时，"故宫"已经在耳朵里了。

实验 2 • 能量 VAD + Smart-Turn 状态机：机器人为什么不打断你

操作： `python vad_smart_turn.py`

目的：复刻上游 VADHandler 两段式判断（Silero 快切 + Smart Turn 裁决）。

实现：numpy 合成"犹豫句"和"结束句"两段波形（正弦谐波+包络），流式状态机对比纯静音计时 vs +Smart 判断。

✓ 实测输出：犹豫句纯 VAD 提交 2 次且"过早打断=是"；加 Smart-Turn 后提交 1 次、不中断；结束句两者均恰好 1 次。代价：确认晚约 0.3~0.4s。

实验 3 • Realtime 协议重放：31 个事件名逐个验明正身

操作： `python realtime_protocol_check.py`

目的：搞懂"假装成 OpenAI"的协议面到底长什么样。

实现：正则扫上游源码提取全部事件名 → 冻结集；重放含工具调用的 24 事件完整一轮；校验成员、方向、收尾顺序；生成延迟账本 metadata。

✓ 实测输出：`31 种事件名` 提取成功；24 事件全部 $\in$ 协议面；`transcript.done < audio.done < response.done` 顺序正确；metadata 190/512 字符合规。

实验 4 • 组件矩阵盘点：768 种组合从哪数出来的

操作：扫描 `repo/src/speech_to_speech/{VAD,STT,LLM,TTS}/` 各目录 handler 文件数。

✓ 实测输出： $VAD\ 2 \times STT\ 12 \times LLM\ 4 \times TTS\ 8 = 768$ 种管线配置——"全模块化"不是口号，是可以数出来的源码事实。

三 • 这个项目给我什么能力

- 🔧 1. 自建隐私语音助手：一台 16GB 内存 Mac 或 24GB 显存 Linux，`pip install speech-to-speech` 三条命令起一个不上云的语音对话服务。
- 🇨🇳 2. 换中文全家桶：STT 槽有 paraformer/sensevoice（中文强项），LLM 槽指向任意国产模型 API，TTS 换中文音色——一套中文语音助手骨架。
- 🤖 3. 给机器人/智能硬件装嘴：Reachy Mini 同款后端；配合 `[[WebRTC机器人应用]]` 的网页遥测面板即可开口说话。
- 🔥 4. 无缝替换托管服务：已有接 OpenAI Realtime 的产品？改个端点地址切自托管，客户端零改造。
- 📚 5. 学架构：这是"生产级 Python 流水线设计"的活教材——队列解耦、BaseHandler 模板方法、注册表模式、全链路延迟埋点，每一样都能搬去别的项目。

四 • 上手三分钟（真机上）

```
# Mac(Apple Silicon, 全本地, 无需API key):  
python3 -m venv .venv && source .venv/bin/activate  
pip install speech-to-speech  
speech-to-speech local --mac-optimal-settings \  
    --model_name mlx-community/Qwen3-4B-Instruct-2507-4bit  
# 说话 → 停顿 → 听回答。Ctrl+C 退出。  
# Linux/NVIDIA: speech-to-speech local --device cuda --llm_backend transformers  
# 只做服务端(无音频): speech-to-speech serve 然后用任意 Realtime 客户端连
```

默认模型约 7.5GB (Parakeet STT + Qwen3-4B LLM + Qwen3-TTS) , 首次运行自动下载。

F:\reminder\s2s-voice-agent · 拾遗学习流水线产出 · 2026-10-03 · 实验 LEX1-4 已真跑通, 数据可复现